

# 中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并行程序设计

批改人：

|       |                                                                          |      |          |
|-------|--------------------------------------------------------------------------|------|----------|
| 实验    | 2-基于MPI的并行矩阵乘法（进阶）                                                       | 专业   | 计算机科学与技术 |
| 学号    | 22336203                                                                 | 姓名   | 沈苇杭      |
| Email | <a href="mailto:shenwh7@mail2.sysu.edu.cn">shenwh7@mail2.sysu.edu.cn</a> | 完成日期 | 2025.4.9 |

## 实验目的

- 掌握MPI集合通信
- 掌握如何使用 `mpi_type_create_struct` 自定义数据类型

## 实验过程与核心代码

自定义数据类型

```
typedef struct {  
    int value;  
    int row;  
    int col;  
} element;
```

```
element my_element;  
MPI_Datatype MPI_element;  
int block_lengths[3] = {1,1,1};  
MPI_Datatype types[3] = {MPI_INT, MPI_INT, MPI_INT};  
MPI_Aint value_address, row_address, col_address;  
MPI_Get_address(&my_element.value, &value_address);  
MPI_Get_address(&my_element.row, &row_address);  
MPI_Get_address(&my_element.col, &col_address);  
MPI_Aint displacement[3] = {0, row_address - value_address, col_address -  
row_address};  
MPI_Type_create_struct(3, block_lengths, displacement, types, &MPI_element);  
MPI_Type_commit(&MPI_element);
```

在使用 `mpi_type_create_struct` 创建数据类型时，需要明确结构体的大小、成员数据类型和成员地址偏移。

## MPI集合通信

```
MPI_Barrier(MPI_COMM_WORLD);
start = MPI_Wtime();

MPI_Scatter(A, row_per_prs * n, MPI_element, Mem_A, row_per_prs * n,
MPI_element, 0, MPI_COMM_WORLD);
MPI_Bcast(B, n * k, MPI_element, 0, MPI_COMM_WORLD);

times(Mem_A, B, Mem_C, m,n,k, row_per_prs);

MPI_Barrier(MPI_COMM_WORLD);
MPI_Gather(Mem_C, row_per_prs * k, MPI_element, C, row_per_prs * k,
MPI_element, 0, MPI_COMM_WORLD);

end = MPI_Wtime();
```

`MPI_Barrier` 用于设置一个“路障”，所有的进程需要在此等待，直到所有进程都执行到这里。

`MPI_Scatter` 用于分发矩阵，给每个进程分发`row_per_prs`行矩阵来计算。

`MPI_Bcast` 用于广播，把矩阵B广播给各个进程。广播的数据量是 `n * k`，数据类型是 `MPI_INT`。

`MPI_Gather` 用于收集各个进程的计算结果。每个进程传输 `row_prs * k` 的数据，数据类型是 `MPI_INT`，即整型。数据存储的缓冲区是 `Mem_C`。

## 矩阵乘法计算

```
void times(element *A, element *B, element *C, int m, int n, int k, int
row_per_prs) {
    for (int i = 0; i < row_per_prs; i++) {
        for (int j = 0; j < k; j++) {
            C[i * k + j].value = 0;
            for (int a = 0; a < n; a++) {
                C[i * k + j].value += A[i * n + a].value * B[a * k + j].value;
            }
        }
    }
}
```

在每个进程中，计算一部分矩阵。

内存管理

```
if (rank == 0) {
    srand(time(NULL));
    A = (element *)malloc(m * n * sizeof(element));
    B = (element *)malloc(n * k * sizeof(element));
    C = (element *)malloc(m * k * sizeof(element));

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            A[i * n + j].value = rand() % 100;
        }
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < k; j++) {
            B[i * k + j].value = rand() % 100;
        }
    }
} else{
    A = (element *)malloc(row_per_prcs * n * sizeof(element));
    B = (element *)malloc(n * k * sizeof(element));
    C = (element *)malloc(row_per_prcs * k * sizeof(element));
}
```

在主进程中，需要分配每个矩阵的内存。在其他进程中，也要分配对应大小的内存。

实验结果

实验结果表格

| 进程数 | 矩阵规模     | 矩阵规模     | 矩阵规模     | 矩阵规模      | 矩阵规模       |
|-----|----------|----------|----------|-----------|------------|
| /   | 128      | 256      | 512      | 1024      | 2048       |
| 1   | 0.016245 | 0.090029 | 0.899509 | 11.117650 | 305.511857 |
| 2   | 0.008624 | 0.078193 | 0.468054 | 5.446493  | 73.421130  |
| 4   | 0.033016 | 0.101327 | 0.570803 | 6.040646  | 84.115925  |
| 8   | 0.020376 | 0.122158 | 0.800787 | 12.768249 | 73.161372  |
| 16  | 0.081303 | 0.335338 | 1.247698 | 7.049079  | 90.211111  |

随着进程数增加，运行时间整体上减少。但由于新建进程、回收进程的开销，进程过多时运行时间反而增大。当矩阵规模较大时，通过适当增加进程，性能有明显的提升，但程序具有系统瓶颈，进程数高于4后，再增加进程数，性能没有明显改善，甚至有降低。因此，程序虽然有较好的扩展性，但仍然需要提升。

实验感想

在本次实验中，我学会了使用 `mpi_type_create_struct` 创建自定义数据类型，以及通过MPI集合通信的函数实现进程间通信。在实验中，我遇到的最大问题依然是 `segmentation fault: address not mapped` 问题，依照上次实验积累的经验进行排查，很快发现了是自定义数据类型过程中的问题，并顺利解决了。